CSE 222A
Winter 2025

Homework 7

Chad Baker, Raphael Huang

March 14, 2025

Overview

Github repo: https://git.ucsc.edu/cse122/w25/clbaker/hw7

The goal of this assignment is to implement and optimize an IBEX 32-bit RISC-V microprocessor using the Openlane2/OpenROAD tool flow. After being given the RTL and memory IP, we were tasked with creating a design setup, a baseline PPA, and making improvements to our baseline design. We will then evaluate this design on three key metrics, Power, Performance, and Area while walking through how we approached each step.

Design Setup

In order to successfully set up our design flow there were four files we needed to create:

- `config.yaml`: Sets up OpenLane flow parameters, allowing us to change things macro placement, power, and other optimizations
- `pin_order.cfg`: Specifies exact I/O placements
- `base.sdc` and `signoff.sdc`: Specifies timing and physical constraints of the desing to be used in static timing analysis

After setting up two macro instances in the config.yaml, we tested our configuration by running the OpenLane flow using:

```
openlane --run-tag latest --overwrite newconfig.yaml
```

Baseline

Before we could successfully run our flow, we created two design constraint files, `base.sdc` and `signoff.sdc`. These files supplied our design with information such as the clock period, input slew, input delay, etc. For both files, we used the values specified in the lab document.

Macro Placement

When trying to run our design, we quickly ran into errors with the placement of the macro instances. They often either overlapped with each other or were placed outside the area of the die, because the dimensions of the core area were allocated automatically during floorplanning according to our `FP_CORE_UTIL`. So, instead of continually guessing the placement, we decided to open up the design in the OpenROAD GUI and see where the best placement of our macros would be.

Our first thought was to place them side-by-side, but this failed for two reasons: congestion and hold slack. With the macros placed so close together it did not leave enough room for the wires to be

routed to where they needed to go. We also realized that it worsened our hold time violations when we placed them so close together. With these constraints figured out, we settled with the placements below:

```
MACROS:
 sky130_sram_1kbyte_1rw1r_32x256_8:
   instances:
     mem_inst0:
       location: [100, 100]
       orientation: "N"
     mem_inst1:
       location: [1000, 1000]
       orientation: "N"
   gds: dir::src/mem/sky130_sram_1kbyte_1rw1r_32x256_8.gds
   lef: dir::src/mem/sky130_sram_1kbyte_1rw1r_32x256_8.lef
   lib:
     "*": dir::src/mem/sky130_sram_1kbyte_1rw1r_32x256_8_TT_1p8V_25C.lib
```

This was the perfect distance for now as it seemed to leave more room for the wires, but it was not enough to make a dent on our hold time, max cap, and max slew violations.

Fixing Hold, Max Cap, and Max Slew Violations

In order to work towards getting rid of the negative hold slack we first settled on two parameters:

```
RUN_POST_GRT_DESIGN_REPAIR: true
RUN_POST_GRT_RESIZER_TIMING: true
```

We decided on `RUN_POST_GRT_DESIGN_REPAIR` because we saw that it was an extra step that would repair the design after Global Placement which could help with slack. In addition, `RUN_POST_GRT_RESIZER_TIMING` was critical to our design as it inserted buffers and resized critical paths. When these two were run on the nominal corner "nom_tt_025C_1v80" this seemed to be enough to fix the negative hold slack, but when run on the more critical corners, specifically "nom_ss_100C_1V60", we were met with the following error:

```
[17:39:05] ERROR   The following error was encountered while running the flow:
                   OpenROAD.RepairDesignPostGPL failed with the following errors:
                   [RSZ-0090] Max transition time from SDC is 0.040ns.
                   Best achievable transition time is 0.043ns with a load of 0.01pF
[17:39:05] ERROR   OpenLane will now quit.
```

In order to combat this error we found a few different fixes. The first was to move the macros further apart and relax `FP_CORE_UTIL`. By doing this we found that the wires were able to find better routes and get rid of this max transition time error. The problem with this fix is that it increased the area of the die. It also showed us that those two commands were not enough to fix the hold time violations for all corners. This brought us to our second solution to the problem which involved the following parameters:

```
GRT_RESIZER_FIX_HOLD_FIRST: true
GRT_DESIGN_REPAIR_MAX_CAP_PCT: 80
GRT_DESIGN_REPAIR_MAX_SLEW_PCT: 80
GRT_DESIGN_REPAIR_MAX_WIRE_LENGTH: 200
```

We chose GRT_RESIZER_FIX_HOLD_FIRST because we had plenty of setup slack (˜40 ns), so we wanted the resizer to focus on fixing our hold time. Next, we found that GRT_DESIGN_REPAIR_MAX_WIRE_LENGTH, GRT_DESIGN_REPAIR_MAX_CAP_PCT, and GRT_DESIGN_REPAIR_MAX_SLEW_PCT were able to get rid of the error from before while also combatting our issues with hold, cap, and slew violations.

GRT_DESIGN_REPAIR_MAX_WIRE_LENGTH helped because it set a max length for wires, which originally defaulted to 600 microns, helping with slack and the max transition time error from before. GRT_DESIGN_REPAIR_MAX_CAP_PCT and GRT_DESIGN_REPAIR_MAX_SLEW_PCT were important because they allowed us to set a target on the max cap and slew violations that were out of control. We tested different permutations with values of [200, 225, 300] for the MAX_WIRE_LENGTH and [25, 50, 75] for MAX_CAP_PCT and MAX_SLEW_PCT.

Corner/Group	Hold Worst Slack	Reg to Reg Paths	Hold TNS	Hold Vio Count	of which reg to reg	Setup Worst Slack	Reg to Reg Paths	Setup TNS	Setup Vio Count	of which reg to reg	Max Cap Violations	Max Slew Violations
Overall	0.2577	0.2577	0.0000	0	0	36.6254	36.6254	0.0000	0	0	61	454
nom_tt_025C_1v80	0.4125	0.4125	0.0000	0	0	43.0680	43.0680	0.0000	0	0	43	30
nom_ss_100C_1v60	0.8790	0.8790	0.0000	0	0	36.9250	36.9250	0.0000	0	0	54	340
nom_ff_n40C_1v95	0.2583	0.2583	0.0000	0	0	45.3927	45.3927	0.0000	0	0	39	24
min_tt_025C_1v80	0.4131	0.4131	0.0000	0	0	43.3192	43.3192	0.0000	0	0	28	24
min_ss_100C_1v60	0.8829	0.8829	0.0000	0	0	37.3612	37.3612	0.0000	0	0	37	334
min_ff_n40C_1v95	0.2577	0.2577	0.0000	0	0	45.5748	45.5748	0.0000	0	0	27	24
max_tt_025C_1v80	0.4115	0.4115	0.0000	0	0	42.8838	42.8838	0.0000	0	0	46	30
max_ss_100C_1v60	0.8700	0.8700	0.0000	0	0	36.6254	36.6254	0.0000	0	0	61	454
max_ff_n40C_1v95	0.2583	0.2583	0.0000	0	0	45.2553	45.2553	0.0000	0	0	45	24

Figure 1: cap_pct=25 slew_pct=25 wire_length=200

Corner/Group	Hold Worst Slack	Reg to Reg Paths	Hold TNS	Hold Vio Count	of which reg to reg	Setup Worst Slack	Reg to Reg Paths	Setup TNS	Setup Vio Count	of which reg to reg	Max Cap Violations	Max Slew Violations
Overall	0.2663	0.2663	0.0000	0	0	37.7613	37.7613	0.0000	0	0	16	46
nom_tt_025C_1v80	0.4162	0.4162	0.0000	0	0	43.7247	43.7247	0.0000	0	0	5	28
nom_ss_100C_1v60	0.8713	0.8713	0.0000	0	0	38.0134	38.0134	0.0000	0	0	8	45
nom_ff_n40C_1v95	0.2663	0.2663	0.0000	0	0	45.8448	45.8448	0.0000	0	0	4	24
min_tt_025C_1v80	0.4204	0.4204	0.0000	0	0	43.9391	43.9391	0.0000	0	0	3	28
min_ss_100C_1v60	0.8840	0.8840	0.0000	0	0	38.3696	38.3696	0.0000	0	0	6	41
min_ff_n40C_1v95	0.2672	0.2672	0.0000	0	0	46.0000	46.0000	0.0000	0	0	2	24
max_tt_025C_1v80	0.4201	0.4201	0.0000	0	0	43.5635	43.5635	0.0000	0	0	12	34
max_ss_100C_1v60	0.8710	0.8710	0.0000	0	0	37.7613	37.7613	0.0000	0	0	16	46
max_ff_n40C_1v95	0.2687	0.2687	0.0000	0	0	45.7321	45.7321	0.0000	0	0	9	24

Figure 2: cap_pct=50 slew_pct=50 wire_length=225

Corner/Group	Hold Worst Slack	Reg to Reg Paths	Hold TNS	Hold Vio Count	of which reg to reg	Setup Worst Slack	Reg to Reg Paths	Setup TNS	Setup Vio Count	of which reg to reg	Max Cap Violations	Max Slew Violations
Overall	0.2524	0.2524	0.0000	0	0	38.4069	38.4069	0.0000	0	0	16	56
nom_tt_025C_1v80	0.4084	0.4084	0.0000	0	0	43.9924	43.9924	0.0000	0	0	6	24
nom_ss_100C_1v60	0.8745	0.8745	0.0000	0	0	38.6441	38.6441	0.0000	0	0	12	43
nom_ff_n40C_1v95	0.2559	0.2559	0.0000	0	0	46.0018	46.0018	0.0000	0	0	6	24
min_tt_025C_1v80	0.4147	0.4147	0.0000	0	0	44.1967	44.1967	0.0000	0	0	2	24
min_ss_100C_1v60	0.8867	0.8867	0.0000	0	0	39.0078	39.0078	0.0000	0	0	5	39
min_ff_n40C_1v95	0.2588	0.2588	0.0000	0	0	46.1510	46.1510	0.0000	0	0	2	24
max_tt_025C_1v80	0.4042	0.4042	0.0000	0	0	43.8248	43.8248	0.0000	0	0	9	24
max_ss_100C_1v60	0.8645	0.8645	0.0000	0	0	38.4069	38.4069	0.0000	0	0	16	56
max_ff_n40C_1v95	0.2524	0.2524	0.0000	0	0	45.8777	45.8777	0.0000	0	0	9	24

Figure 3: cap_pct=50 slew_pct=50 wire_length=300

As you can see from the data, it seemed like this smaller max wire lengths corresponded to more hold slack, and the greater the max cap and max slew percentage the less max cap and max slew errors we had. After more testing we landed on our final values of:

```
GRT_DESIGN_REPAIR_MAX_CAP_PCT: 80
GRT_DESIGN_REPAIR_MAX_SLEW_PCT: 80
GRT_DESIGN_REPAIR_MAX_WIRE_LENGTH: 200
```

Fixing Antenna Violations

The last thing we fixed were the antenna violations. We first tried using:

```
GRT_ANTENNA_ITERS: 15
GRT_ANTENNA_MARGIN: 50
```

But this took up a lot of time (more than 2 hours!) and would lead to inconsistent results with still too many antenna errors. We then tried

```
RUN_HEURISTIC_DIODE_INSERTION: true
```

which brought our antenna errors all the way down to 3.

Baseline Results

We inspected our `results.json` file as well as certain `.rpt` and timing summary files to find the following results for the final signoff of our baseline configuration.

First, setup and hold violations:

```
"timing__hold__ws": 0.2845048721583702,
"timing__setup__ws": 35.945832217096566,
"timing__hold__tns": 0,
"timing__setup__tns": 0,
"timing__hold__wns": 0,
"timing__setup__wns": 0,
```

Zero total negative slack for setup and hold for all timing corners shows that we successfully eliminated all setup and hold violations in our baseline configuration.

Next, max capacitance violations:

```
"design__max_cap_violation__count__corner:nom_tt_025C_1v80": 0,
"design__max_cap_violation__count__corner:min_tt_025C_1v80": 0,
"design__max_cap_violation__count__corner:min_ss_100C_1v60": 0,
"design__max_cap_violation__count": 0,
```

and max slew violations:

```
"design__max_slew_violation__count__corner:nom_tt_025C_1v80": 24,
"design__max_slew_violation__count__corner:min_tt_025C_1v80": 24,
"design__max_slew_violation__count__corner:min_ss_100C_1v60": 24,
"design__max_slew_violation__count": 24,
```

We successfully removed all max capacitance violations, but it was extremely difficult to root out all the max slew violations. We decided to take a look at the `checks.rpt` for our nom_tt_025C_1v80 corner to see what pins were causing these violations and how severe they were, and it looked like this:

```
288
289   ============================================================================
290    report_check_types -max_slew -max_cap -max_fanout -violators
291   ============================================================================
292   ======================== max_tt_025C_1v80 Corner ===================================
293
294   max slew
295
296   Pin                                      Limit        Slew       Slack
297   ---------------------------------------------------------------------------
298   mem_inst1/addr0[2]                     0.040000    0.473073   -0.433073 (VIOLATED)
299   mem_inst1/addr0[3]                     0.040000    0.422714   -0.382714 (VIOLATED)
300   mem_inst0/addr1[5]                     0.040000    0.402447   -0.362447 (VIOLATED)
301   mem_inst1/addr0[4]                     0.040000    0.400232   -0.360231 (VIOLATED)
302   mem_inst0/addr1[6]                     0.040000    0.389589   -0.349589 (VIOLATED)
303   mem_inst1/addr0[5]                     0.040000    0.379755   -0.339755 (VIOLATED)
304   mem_inst0/addr1[4]                     0.040000    0.379681   -0.339681 (VIOLATED)
305   mem_inst1/addr0[6]                     0.040000    0.328316   -0.288316 (VIOLATED)
306   mem_inst0/addr1[3]                     0.040000    0.259490   -0.219490 (VIOLATED)
307   mem_inst0/addr1[2]                     0.040000    0.257993   -0.217993 (VIOLATED)
308   mem_inst1/addr0[7]                     0.040000    0.212835   -0.172835 (VIOLATED)
309   mem_inst0/addr0[3]                     0.040000    0.203073   -0.163073 (VIOLATED)
310   mem_inst0/addr0[6]                     0.040000    0.186498   -0.146498 (VIOLATED)
311   mem_inst0/addr0[5]                     0.040000    0.184359   -0.144359 (VIOLATED)
312   mem_inst0/addr0[4]                     0.040000    0.182126   -0.142126 (VIOLATED)
313   mem_inst0/addr0[7]                     0.040000    0.180949   -0.140949 (VIOLATED)
314   mem_inst1/addr1[2]                     0.040000    0.158654   -0.118654 (VIOLATED)
315   mem_inst0/addr1[7]                     0.040000    0.158587   -0.118587 (VIOLATED)
316   mem_inst0/addr0[2]                     0.040000    0.150258   -0.110258 (VIOLATED)
317   mem_inst1/addr1[3]                     0.040000    0.142351   -0.102351 (VIOLATED)
318   mem_inst1/addr1[6]                     0.040000    0.113866   -0.073866 (VIOLATED)
319   mem_inst1/addr1[5]                     0.040000    0.111616   -0.071616 (VIOLATED)
320   mem_inst1/addr1[4]                     0.040000    0.101850   -0.061850 (VIOLATED)
321   mem_inst1/addr1[7]                     0.040000    0.090660   -0.050660 (VIOLATED)
322
```

Figure 4: checks.rpt, nom_tt_025C_1v80, post-PNR STA

We were shocked to find that the violations were so bad, but after a bit of thought we realized that the high fanout/capacitive load of a memory macro's address input net likely made it impossible for us to reduce the slew to below the limit. Plus, high slew on the address input to memory would affect the speed of the chip, but not the functionality. Therefore, we decided to move on to our optimizations instead of focusing on these violations.
Finally, we took a look at the antenna violations:

```
"antenna__violating__nets": 3,
"antenna__violating__pins": 3,
"route__antenna_violation__count": 3,
```

Although it is bad to have any antenna violations at all, we had reduced it from hundreds of violations to a handful, and decided it was good enough to move on.

Baseline PPA Metrics

Power:


```
"power__internal__total": 0.011125561781227589,
"power__switching__total": 0.02109256014227867,
"power__leakage__total": 3.84042359655723e-05,
"power__total": 0.03225652500987053,
```


Performance: 100ns clock period, no timing violations

Area:


```
"design__die__area": 2712760,
"design__core__area": 2656120,
```


Optimization 1: Performance

For our first improvement we decided to target clock period and clock frequency. Our initial clock period was **100ns**. To start we just started playing with the clock period setting it lower and lower in both the `config.yaml` and the two `.sdc` files. We also found that you could find what the program thought the minimum clock period could be when looking in the `clock.rpt` file in your STAPostPNR step. By using the value specified by the worst corner we found that we could safely decrease our clock frequency to about 45ns.
Although this was a great improvement and showed how much slack our baseline model had left over, we felt there was still more room to improve. We started looking into possible ways to bring it lower and landed on a few changes to the functionality of Yosys in our flow. We added the following to our config:

```
SYNTH_SIZING: true
SYNTH_STRATEGY: 'DELAY 3'
```

`SYNTH_SIZING` enables `abc`, the logic synthesis tool, to perform cell sizing optimizations BEFORE global placement. `SYNTH_STRATEGY` is a enum that changes the strategy `abc` uses to perform logic synthesis and technology mapping. The possible options are `'AREA 0''AREA 1''AREA 2''AREA 3''DELAY 0''DELAY 1''DELAY 2''DELAY 3''DELAY 4'`. The `AREA` syntheiszes to reduce area, and `DELAY` synethsizes to try and focus on timing. There are multiple different strategies for each of those goals, but there is no way to tell which strategy is best for your design without simply guessing and checking.
This improvement gave us even more leeway for our hold and setup slack and allowed us to bring our clock period all the way down to **38 ns**. You can see the difference in using these commands in the figures below.

Corner/Group	Hold Worst Slack	Reg to Reg Paths	Hold TNS	Hold Vio Count	of which reg to reg	Setup Worst Slack	Reg to Reg Paths	Setup TNS	Setup Vio Count	of which reg to reg	Max Cap Violations	Max Slew Violations
Overall	-0.0074	0.0076	-0.0074	1	0	-0.5426	-0.5426	-2.2835	10	10	0	24
nom_tt_025C_1v80	0.2665	0.2666	0.0000	0	0	12.2927	12.2927	0.0000	0	0	0	24
nom_ss_100C_1v60	0.8625	0.7268	0.0000	0	0	0.5734	0.5734	0.0000	0	0	0	24
nom_ff_n40C_1v95	0.0923	0.0923	0.0000	0	0	14.5008	14.5008	0.0000	0	0	0	23
min_tt_025C_1v80	0.2633	0.2633	0.0000	0	0	12.4952	12.4952	0.0000	0	0	0	24
min_ss_100C_1v60	0.2120	0.7213	0.0000	0	0	1.6652	1.6652	0.0000	0	0	0	24
min_ff_n40C_1v95	0.0909	0.0909	0.0000	0	0	14.6479	14.6479	0.0000	0	0	0	23
max_tt_025C_1v80	0.2622	0.2622	0.0000	0	0	12.0989	12.0989	0.0000	0	0	0	24
max_ss_100C_1v60	-0.0074	0.7319	-0.0074	1	0	-0.5426	-0.5426	-2.2835	10	10	0	24
max_ff_n40C_1v95	0.0076	0.0076	0.0000	0	0	14.3484	14.3484	0.0000	0	0	0	23

Figure 5: Before synthesis config changes

Corner/Group	Hold Worst Slack	Reg to Reg Paths	Hold TNS	Hold Vio Count	of which reg to reg	Setup Worst Slack	Reg to Reg Paths	Setup TNS	Setup Vio Count	of which reg to reg	Max Cap Violations	Max Slew Violations
Overall	0.0155	0.0867	0.0000	0	0	4.6947	4.6947	0.0000	0	0	0	24
nom_tt_025C_1v80	0.2637	0.2637	0.0000	0	0	12.8895	12.8895	0.0000	0	0	0	24
nom_ss_100C_1v60	0.1146	0.7943	0.0000	0	0	6.0588	6.0588	0.0000	0	0	0	24
nom_ff_n40C_1v95	0.0901	0.0901	0.0000	0	0	14.8836	14.8836	0.0000	0	0	0	23
min_tt_025C_1v80	0.2624	0.2624	0.0000	0	0	13.0710	13.0710	0.0000	0	0	0	24
min_ss_100C_1v60	0.2212	0.7522	0.0000	0	0	7.2631	7.2631	0.0000	0	0	0	24
min_ff_n40C_1v95	0.0886	0.0886	0.0000	0	0	14.9987	14.9987	0.0000	0	0	0	23
max_tt_025C_1v80	0.2656	0.2656	0.0000	0	0	12.7096	12.7096	0.0000	0	0	0	24
max_ss_100C_1v60	0.0155	0.7576	0.0000	0	0	4.6947	4.6947	0.0000	0	0	0	24
max_ff_n40C_1v95	0.0867	0.0867	0.0000	0	0	14.7710	14.7710	0.0000	0	0	0	23

Figure 6: After synthesis config changes

Optimization 2: Area

For our second improvement, we decided to target the core area of the design. Our baseline configuration area was $2712760\mu m^2$, but after the first optimization it had ballooned all the way up to $3721390\mu m^2$. First, we had to constain the exact dimensions of the die area instead of letting the floorplanning step automatically create the dimensions implied by our `FP_CORE_UTIL`. To do this, we specified the exact area using:

```
DIE_AREA: [0, 0, 1000, 1800]
```

Next, we looked at the OpenROAD gui and saw that we had to move the memory macros to decrease the area:

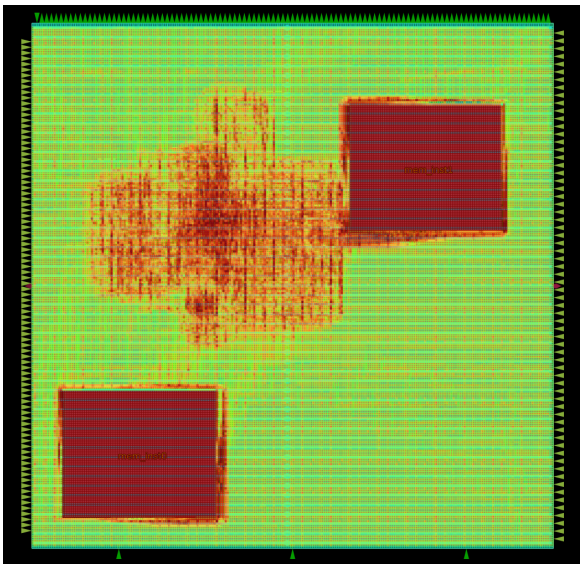

Figure 7: Baseline macro placement

We moved the macros to be stacked on top of each other to save horizontal space, and we flipped `mem_inst1` vertically as we saw that many nets that came out of that instance would be closer to their destinations if the pins were on the other side.

```
MACROS:
  sky130_sram_1kbyte_1rw1r_32x256_8:
    gds: dir::src/mem/sky130_sram_1kbyte_1rw1r_32x256_8.gds
    lef: dir::src/mem/sky130_sram_1kbyte_1rw1r_32x256_8.lef
    lib:
      "*": dir::src/mem/sky130_sram_1kbyte_1rw1r_32x256_8_TT_1p8V_25C.lib
```

```
  instances:
    mem_inst0:
      location: [200, 100]
      orientation: "N"
    mem_inst1:
      location: [300, 1300]
      orientation: "S"
```

And, finally, to further shrink area as we still saw space for improvement, we used:

```
PL_MACRO_HALO: "{25} {25}"
GRT_ALLOW_CONGESTION: True
GRT_ADJUSTMENT: 0.2
```

`PL_MACRO_HALO` creates a ring around the memory macros that are used for routing the inputs and outputs of that macro, which had minimal impact but might've allowed us to decrease our area without increasing congestion so much that the flow would error out. `GRT_ALLOW_CONGESTION` gives the congestion data from the mid-PNR STA to the global routing stages, so it can take it into account. `GRT_ADJUSTMENT` "reduces the routing capacity of the edges between the cells in the global routing graph for all layers" (Source: OpenLane2 Docs), which was defaults to 0.3. Setting it to 0.2 gave us better timing results, which allowed us to reduce the area of the chip further.
Our final design, after optimizations 1 and 2, looks like this:

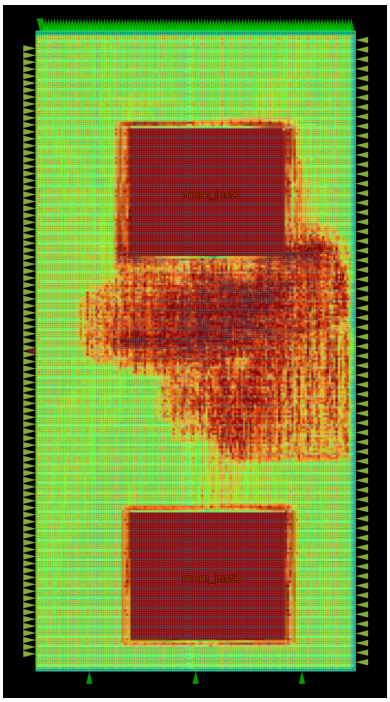

Figure 8: Final design in OpenROAD GUI

Conclusions

a) What are your final improved PPA metrics?

INITIAL

Power:


```
"power__internal__total": 0.011125561781227589,
"power__switching__total": 0.02109256014227867,
"power__leakage__total": 3.84042359655723e-05,
"power__total": 0.03225652500987053,
```


Performance: **100ns** clock period, no timing violations

Area:


```
"design__die__area": 2712760,
"design__core__area": 2656120,
```


AFTER OPTIMIZATION 1

Power:


```
"power__internal__total": 0.028430888429284096,
"power__switching__total": 0.055053625255823135,
"power__leakage__total": 3.8522000977536663e-05,
"power__total": 0.08352303504943848,
```


Performance: **38ns** clock period, no timing violations

Area:


```
"design__die__area": 3795200,
"design__core__area": 3730190,
```


FINAL

Power:


```
"power__internal__total": 0.026507217437028885,
"power__switching__total": 0.06402982026338577,
"power__leakage__total": 3.835805182461627e-05,
"power__total": 0.09057539701461792,
```


Performance: **38ns** clock period, 1 timing violation (max_ss_100C_1v60 setup time)

Area:


```
"design__die__area": 1800000,
"design__core__area": 1755810,
```

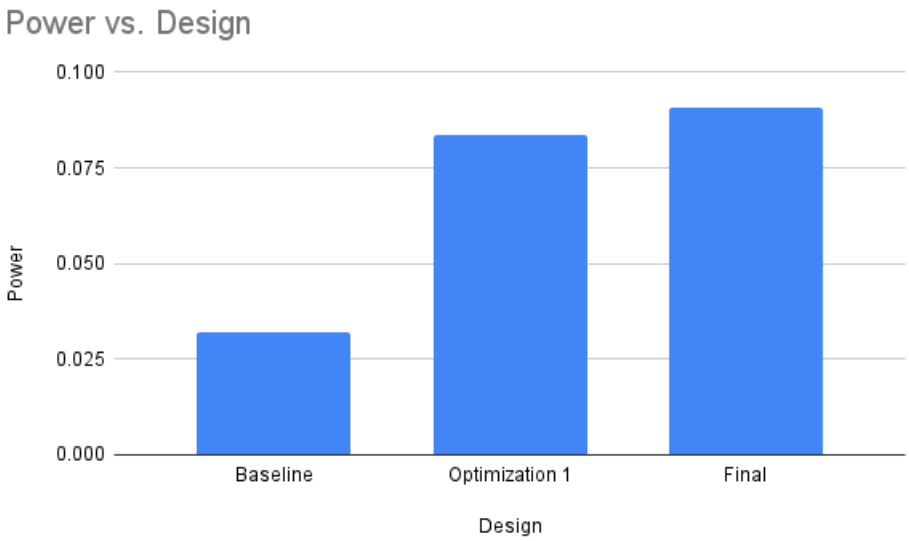

Figure 9: Power (W) vs. Design

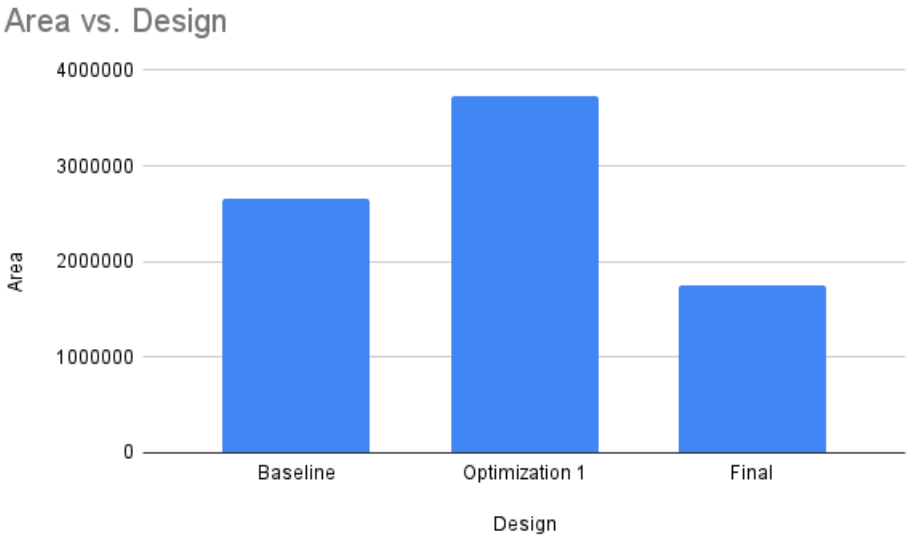

Figure 10: Area (micron^2) vs. Design

b) Describe the Power breakdown. How much power do the sequential and combinational gates take in your design?

In our final design, the majority of our power is consumed by the internal power (1.91e-02) and switching power (4.41e-02) of the combinational gates. 89.4% of the power is consumed by combinational gates.

Group	Internal (W)	Switching (W)	Leakage (W)	Total Power (W)	Percentage (%)
Sequential	2.90e-03	8.87e-04	2.31e-08	3.79e-03	5.2%
Combinational	1.87e-02	4.83e-02	6.86e-08	6.70e-02	91.9%
Clock	8.44e-04	1.26e-03	9.34e-08	2.10e-03	2.9%
Macro	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Pad	0.00e+00	0.00e+00	0.00e+00	0.00e+00	0.0%
Total	2.25e-02	5.04e-02	1.85e-07	7.29e-02	100.0%

c) What is your critical path and why is it critical? For example: How many stages is it? Does it have long wires (show an image of it)? Does it have high fanout?
Our critical path was a long chain of combinational logic that travelled a long way across our chip, causing the PNR algorithm to place more than 20 buffers (!) to drive the capactive load of such complex logic, at the cost of timing.

```
Startpoint: _26888_ (rising edge-triggered flip-flop clocked by clk_i)
Endpoint: _26984_ (rising edge-triggered flip-flop clocked by clk_i)
Path Group: clk_i
Path Type: max

  Delay    Time   Description
---------------------------------------------------------
   0.00    0.00   clock clk_i (rise edge)
   1.76    1.76   clock network delay (propagated)
   0.00    1.76 ^ _26888_/CLK (sky130_fd_sc_hd__dfxtp_1)
   0.42    2.18 ^ _26888_/Q (sky130_fd_sc_hd__dfxtp_1)
   0.29    2.47 ^ wire4778/X (sky130_fd_sc_hd__buf_4)
   0.18    2.65 ^ max_cap758/X (sky130_fd_sc_hd__buf_1)
   0.26    2.91 ^ wire4425/X (sky130_fd_sc_hd__clkbuf_8)
   0.23    3.14 ^ max_cap756/X (sky130_fd_sc_hd__buf_6)
   0.21    3.34 ^ max_cap755/X (sky130_fd_sc_hd__buf_6)
   0.28    3.62 ^ wire4086/X (sky130_fd_sc_hd__clkbuf_8)
   0.33    3.95 ^ max_length4085/X
... (excluded for brevity)
   0.30   15.14 v wire1179/X (sky130_fd_sc_hd__clkbuf_2)
   0.25   15.40 v wire1178/X (sky130_fd_sc_hd__clkbuf_2)
   0.29   15.69 v wire1177/X (sky130_fd_sc_hd__clkbuf_8)
   0.29   15.98 v wire1176/X (sky130_fd_sc_hd__buf_4)
   0.11   16.09 ^ _17022_/Y (sky130_fd_sc_hd__nand2_1)
   0.27   16.36 ^ wire1100/X (sky130_fd_sc_hd__buf_6)
   0.46   16.81 ^ wire1099/X (sky130_fd_sc_hd__buf_2)
   0.27   17.09 ^ wire1098/X (sky130_fd_sc_hd__buf_4)
```

```
   0.13   17.22 v _17023_/Y (sky130_fd_sc_hd__nand4_1)
   0.27   17.49 v _17024_/X (sky130_fd_sc_hd__o22a_2)
   0.28   17.77 v _17026_/X (sky130_fd_sc_hd__o21a_1)
   0.24   18.01 ^ _17037_/Y (sky130_fd_sc_hd__a21oi_1)
   0.26   18.27 ^ wire895/X (sky130_fd_sc_hd__buf_1)
   0.22   18.49 ^ _17041_/X (sky130_fd_sc_hd__o22a_1)
   0.12   18.61 v _17044_/Y (sky130_fd_sc_hd__o2bb2ai_2)
   0.19   18.80 v max_cap226/X (sky130_fd_sc_hd__clkbuf_2)
   0.19   18.99 v wire225/X (sky130_fd_sc_hd__clkbuf_2)
   0.26   19.25 ^ _17045_/Y (sky130_fd_sc_hd__nor2_4)
   0.34   19.59 ^ max_cap179/X (sky130_fd_sc_hd__clkbuf_4)
   0.15   19.74 v _17065_/Y (sky130_fd_sc_hd__nand2_2)
   0.37   20.11 ^ _24792_/X (sky130_fd_sc_hd__mux2_8)
   0.32   20.43 ^ wire28/X (sky130_fd_sc_hd__clkbuf_8)
   0.25   20.68 ^ max_length27/X (sky130_fd_sc_hd__buf_6)
   0.36   21.04 v _24804_/X (sky130_fd_sc_hd__mux2_1)
   0.00   21.04 v _26984_/D (sky130_fd_sc_hd__dfxtp_1)
          21.04   data arrival time

  38.00   38.00   clock clk_i (rise edge)
   1.62   39.62   clock network delay (propagated)
  -0.20   39.42   clock uncertainty
   0.00   39.42   clock reconvergence pessimism
          39.42 ^ _26984_/CLK (sky130_fd_sc_hd__dfxtp_1)
  -0.11   39.32   library setup time
          39.32   data required time
-----------------------------------------------------------
          39.32   data required time
         -21.04   data arrival time
-----------------------------------------------------------
          18.27   slack (MET)
```

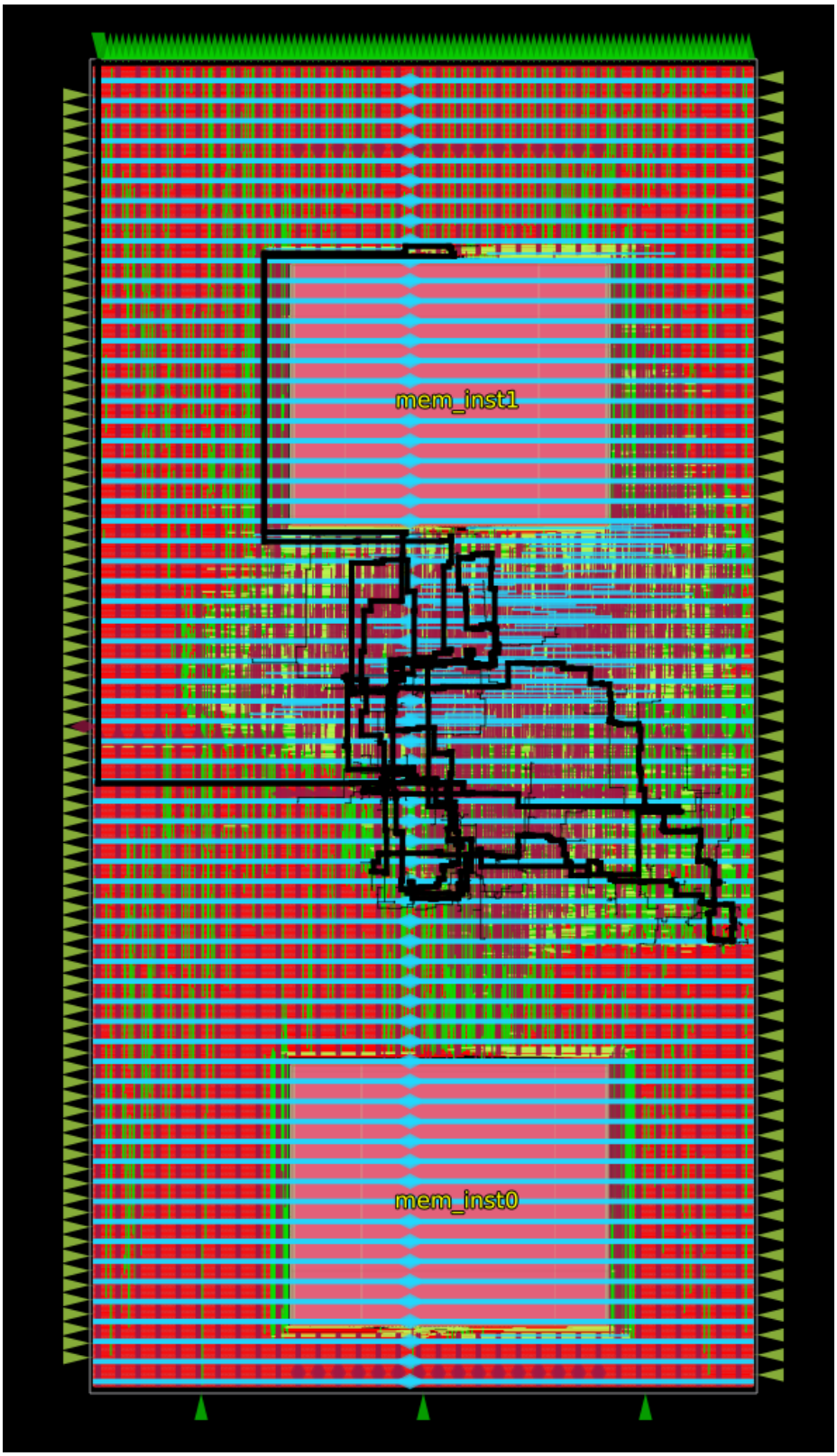

Figure 11: Critical path in OpenROAD GUI

d) What is your maximum clock skew? How much clock skew is on your critical path?

From `results.json`:


```
"clock__skew__worst_hold": -0.5193741139726124,
"clock__skew__worst_setup": 0.5031720738045201,
```


As described in **Fixing Hold, Max Cap, and Max Slew Violations**, our max slew resulted from the memory address input. Our maximum clock skew was 0.429779ns, and our critical path had a clock skew of 0.331ns.

e) How big is your design floorplan? What is your utilization?
Our final design floorplan size was $1800000\mu m^2$ with a core area of $1755810\mu m^2$. Our final design utilization was 35.3%, which leaves 64.7% of the core for routing.

f) What is the wirelength of your placement?

The estimated final wirelength of our design was $1257480\mu m^2$.

g) Where is the maximum congestion in your design? Physically show this in the GUI.

Please refer back to Figure 8. We can see that the maximum congestion is in the middle of the two instances of the core.

h) Write a narrative of your approach to improve PPA. What did each option intend to improve and what were the results?

Please see previous sections.

i) Did improving one metric hurt another?

Yes, evidently it did. When we tried to increase performance, we had to do it at the cost of area massively increasing. Then, when we decreased area, a slower clock period would've been necessary to eliminate the setup time violations. Finally, power increased throughout both of our optimizations.